Chapter 2

Branching, merging, rebasing

Quan Nguyen
quannd@ptit.edu.vn

August 2026

Table of Contents

1. Branching
2. Branch Management
3. Remote Branches
4. Rebasing

Table of Contents

1. Branching
2. Branch Management
3. Remote Branches
4. Rebasing

Creating a Branch

- Command: `git branch <branchname>`
- Creates a new pointer to the current commit
- Does **not** switch you to that branch automatically

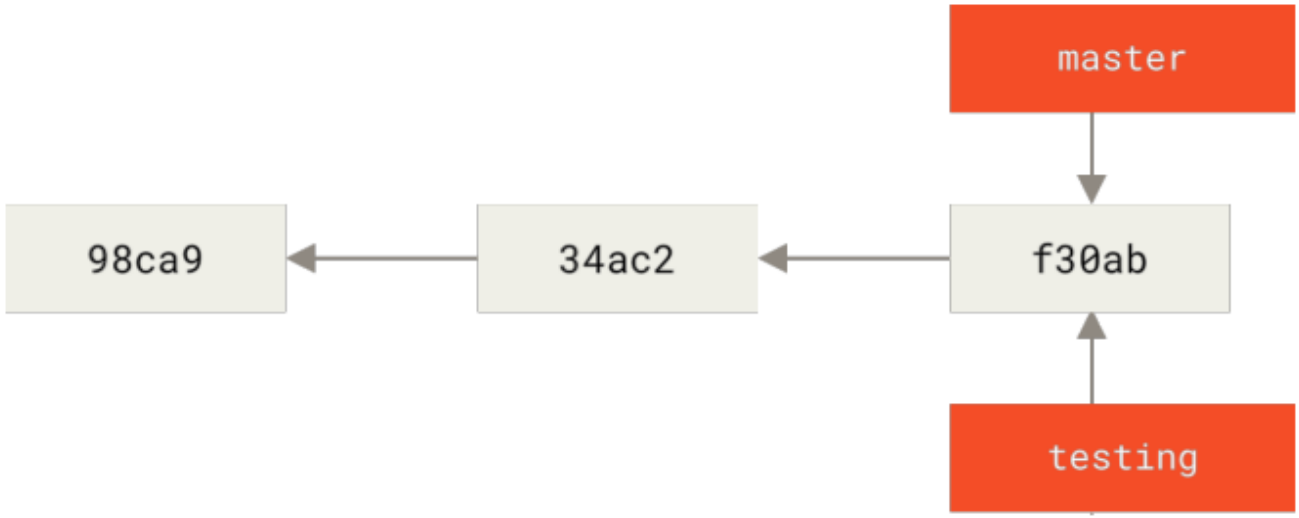


Figure 12. Two branches pointing into the same series of commits

Switching Branches

- Command: `git checkout <branchname>`
- Moves `HEAD` to point at the given branch
- Updates working directory to match that branch's snapshot

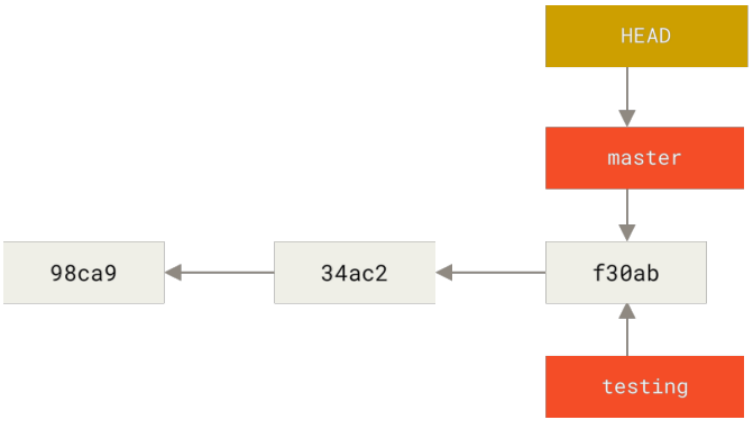


Figure 13. HEAD pointing to a branch

$ git log –oneline –decorate
f30ab (HEAD - master, testing) Add feature #32 - ability to add new formats to the central interface
34ac2 Fix bug #1328 - stack overflow under certain conditions
98ca9 Initial commit

Switching Branches

- Command: `git checkout testing`



Figure 14. HEAD points to the current branch

Switching Branches

- Command: `$ vim test.rb`
- Command: `$ git commit -a -m 'Make a change'`

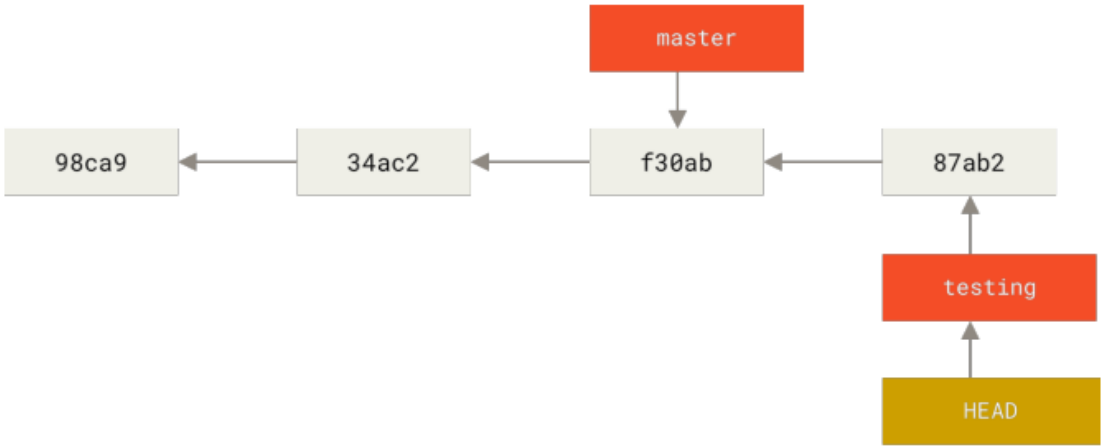


Figure 15. The HEAD branch moves forward when a commit is made

what happen if we run `$ git checkout master`?

Switching Branches

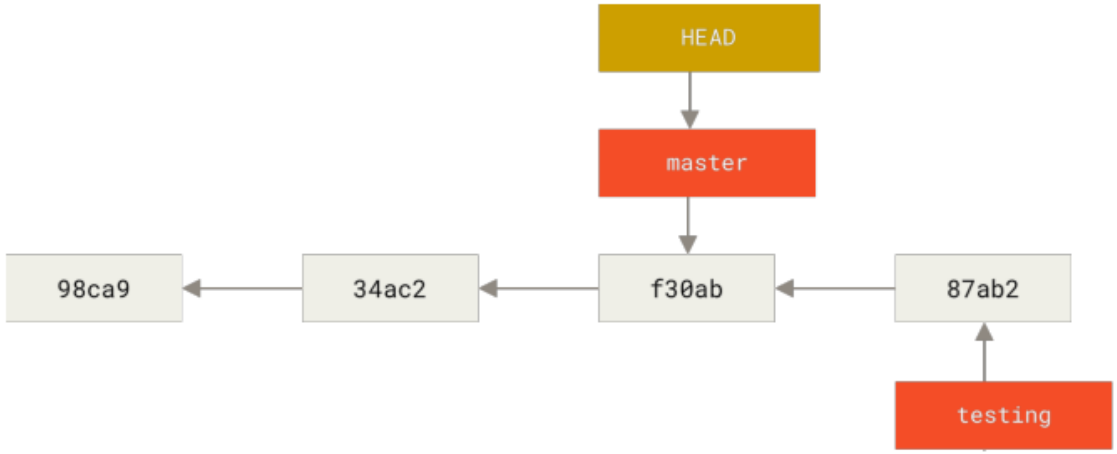


Figure 16. HEAD moves when you checkout

What if:

$ vim test.rb

$ git commit -a -m 'Make other changes'

Switching Branches

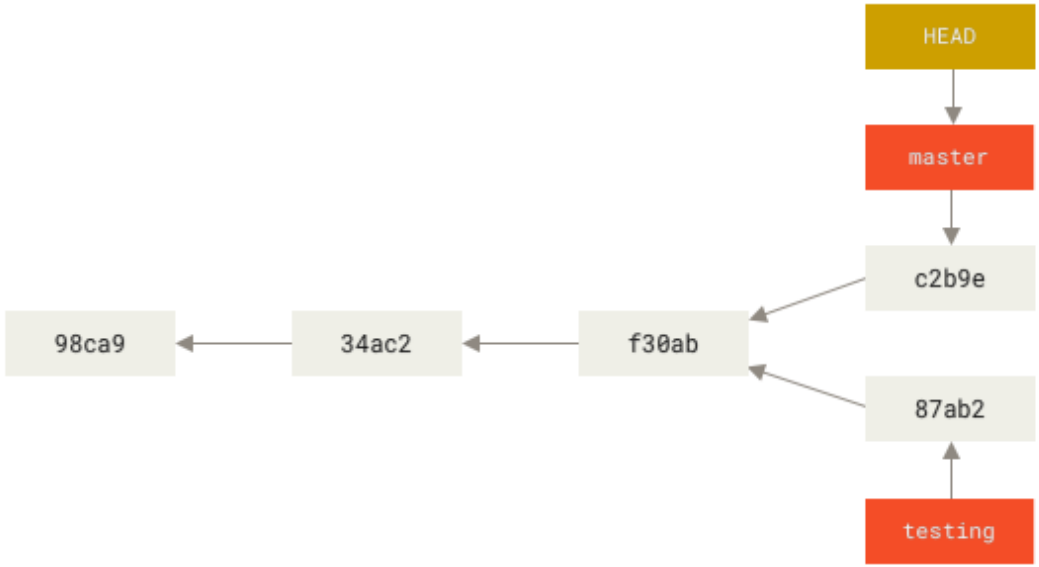


Figure 17. Divergent history

Create and Switch in One Step

- Creates the branch and switches to it immediately
- Modern alternative: `git switch -c <newbranchname>`

The `git switch` Command (Git 2.23+)

- `git switch <branch>` → switch to an existing branch
- `git switch -c <branch>` → create and switch (`-c` = create)
- `git switch -` → return to the previous branch

Real-World Branching Workflow

1. Work on a website (on `master`)
2. Create a branch for a new user story
3. Do some work in that branch
4. An urgent issue appears $\rightarrow$ need a hotfix

Handling the Hotfix Interruption

1. Switch to the production branch
2. Create a branch to add the hotfix
3. Test, then merge the hotfix branch, and push to production
4. Switch back to the original user-story branch and continue

Example: page 70

Merging: Three-Way Merge

- Occurs when both branches have diverged
- Git uses the two branch tips plus their common ancestor
- Produces a new merge commit with two parents

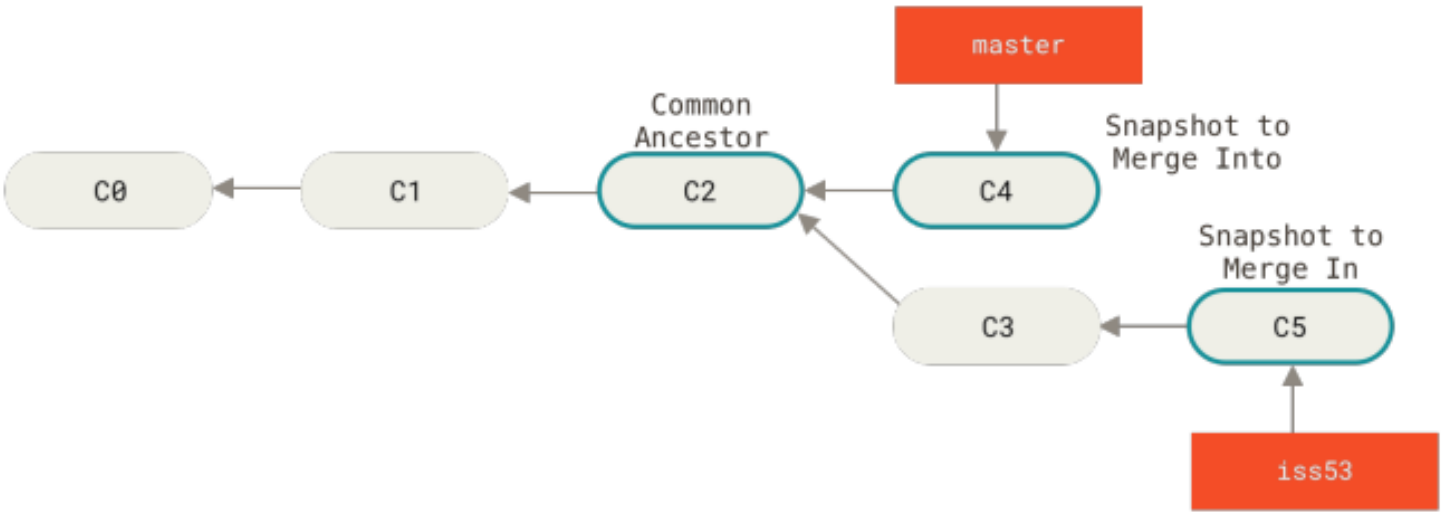


Figure 24. Three snapshots used in a typical merge

Merging: Three-Way Merge

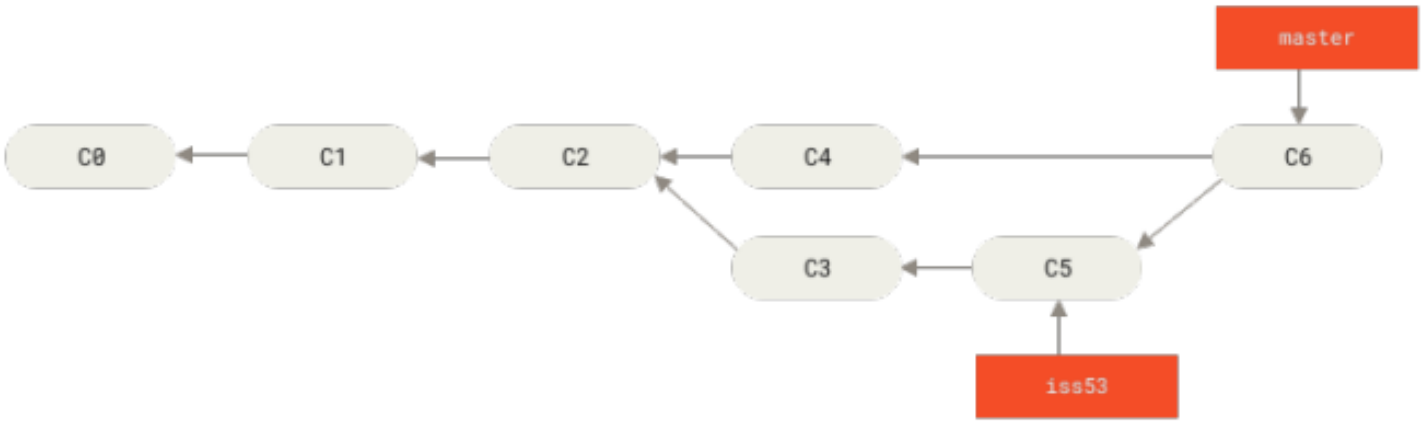


Figure 25. A merge commit

Table of Contents

1. Branching
2. Branch Management
3. Remote Branches
4. Rebasing

Session Objectives

- List branches and identify the current one
- View the last commit on each branch
- Filter branches by merge status (`--merged` / `--no-merged`)
- Safely delete merged branches; force-delete unmerged ones
- Rename a branch, locally and on the remote
- Understand the risks of renaming a default branch (`master`/`main`)

Listing Branches

- Command: `git branch` (no arguments)
- Lists all local branches
- The $*$ prefix marks the currently checked-out branch (where `HEAD` points)
- Committing now moves that starred branch forward

Example: `git branch`

```
$ git branch
  iss53
* master
  testing
```

- `master` is currently checked out
- `iss53` and `testing` are other local branches

Viewing Last Commit per Branch

- Command: `git branch -v`
- Shows the most recent commit on each branch
- Useful for a quick snapshot of branch progress

Example: git branch -v

```
$ git branch -v
  iss53   93b412c Fix javascript issue
* master  7a98805 Merge branch 'iss53'
  testing 782fd34 Add scott to the author list
```

Filtering by Merge Status

- `--merged` and `--no-merged` filter branches by merge state
- Relative to the branch you currently have checked out
- Helps identify branches that are safe (or unsafe) to delete

Branches Already Merged

- Command: `git branch --merged`

```
$ git branch --merged
  iss53
* master
```

- `iss53` appears because its work was already merged in
- Branches without `*` here are generally safe to delete

Deleting Merged Branches

- Command: `git branch -d <branch>`
- Safe because the branch's work is already incorporated elsewhere
- No commits are lost by deleting it

Branches Not Yet Merged

- Command: `git branch --no-merged`

```
$ git branch --no-merged
  testing
```

- `testing` contains work not yet merged into the current branch

Deletion Is Blocked for Unmerged Work

```
$ git branch -d testing
error: The branch 'testing' is not
fully merged.
If you are sure you want to delete
it, run 'git branch -D testing'.
```

- Git protects you from accidentally losing unmerged commits

Force-Deleting a Branch

- Command: `git branch -D <branch>`
- Overrides the safety check
- Deletes the branch and discards its unmerged work permanently

Renaming a Branch — Caution

- Do not rename branches still in use by collaborators
- Renaming `master`/`main`/mainline branches needs special care
- Coordinate with the team before renaming any shared branch

Renaming a Branch Locally

- Goal: rename `bad-branch-name` to `corrected-branch-name`
- Command: `git branch --move bad-branch-name corrected-branch-name`
- History is fully preserved
- Change is local only at this point

Publishing the Renamed Branch

- Command: `git push --set-upstream origin corrected-branch-name`
- Makes the corrected name visible to collaborators on the remote
- The old name still exists on the remote until explicitly removed

Cleaning Up the Old Remote Branch

- Command: `git push origin --delete bad-branch-name`
- Removes the old branch name from the remote
- Completes the rename: old name fully replaced

Renaming a Default Branch — Warning

- Renaming `master`/`main`/default branches can break:
  - Integrations and CI/CD services
  - Helper utilities and build/release scripts
- Always consult collaborators first
- Search the repo for hardcoded references to the old name

Example: `master` → `main`

- Rename locally: `git branch --move master main`
- Push and set upstream: `git push --set-upstream origin main`
- Local `master` is gone; `main` now exists locally and remotely
- The old `master` branch still remains on the remote

Completing the Default Branch Transition

- Update dependent projects' code/configuration
- Update test-runner configuration files
- Adjust build and release scripts
- Redirect the repo host's default branch and merge rules
- Update documentation references
- Close or merge pull requests targeting the old branch

Final Cleanup

- Once `main` is verified to perform identically to `master`
- Delete the old branch from the remote:
- Command: `git push origin --delete master`

Table of Contents

1. Branching
2. Branch Management
3. Remote Branches
4. Rebasing

Session Objectives

- Understand remote references and remote-tracking branches
- Push local branches to a remote
- Set up and inspect tracking branches
- Pull changes from a remote (fetch + merge)
- Delete remote branches

Remote References

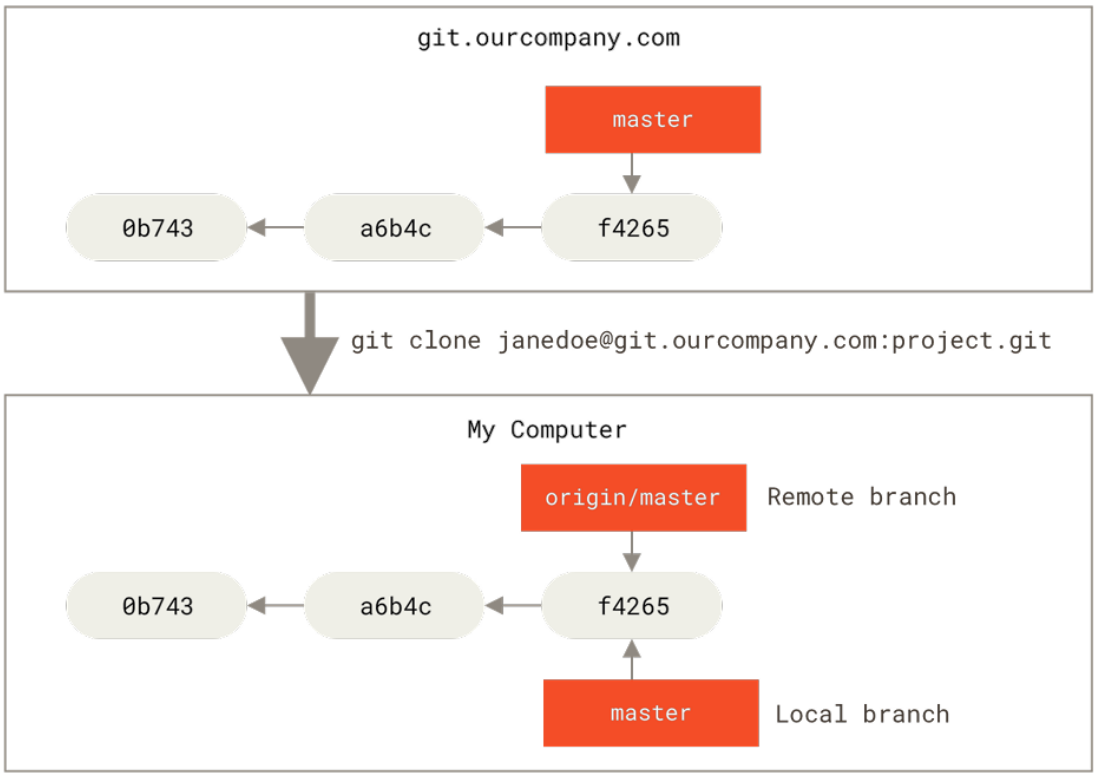
- Remote references = pointers in remote repos (branches, tags, etc.)
- List them explicitly: `git ls-remote <remote>`
- Or with more detail: `git remote show <remote>`
- More common approach: remote-tracking branches

What Is a Remote-Tracking Branch?

- Local references to the state of remote branches
- You cannot move them manually — Git moves them on network operations
- Think of them as bookmarks for "where the remote was last time"
- Naming form: `<remote>/<branch>` (e.g., `origin/master`)

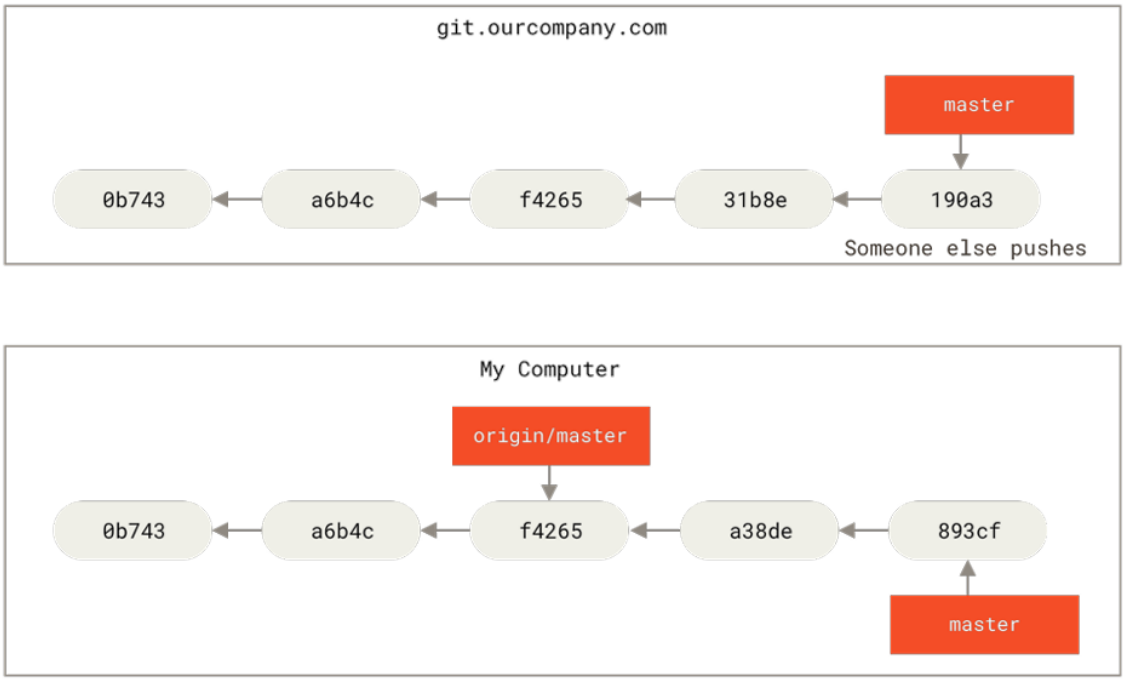
Example: Cloning Sets Up Tracking

- Clone from `git.ourcompany.com`
- Git names the remote `origin` automatically
- Creates `origin/master` pointing to the remote's `master`
- Creates a local `master` branch starting at the same commit



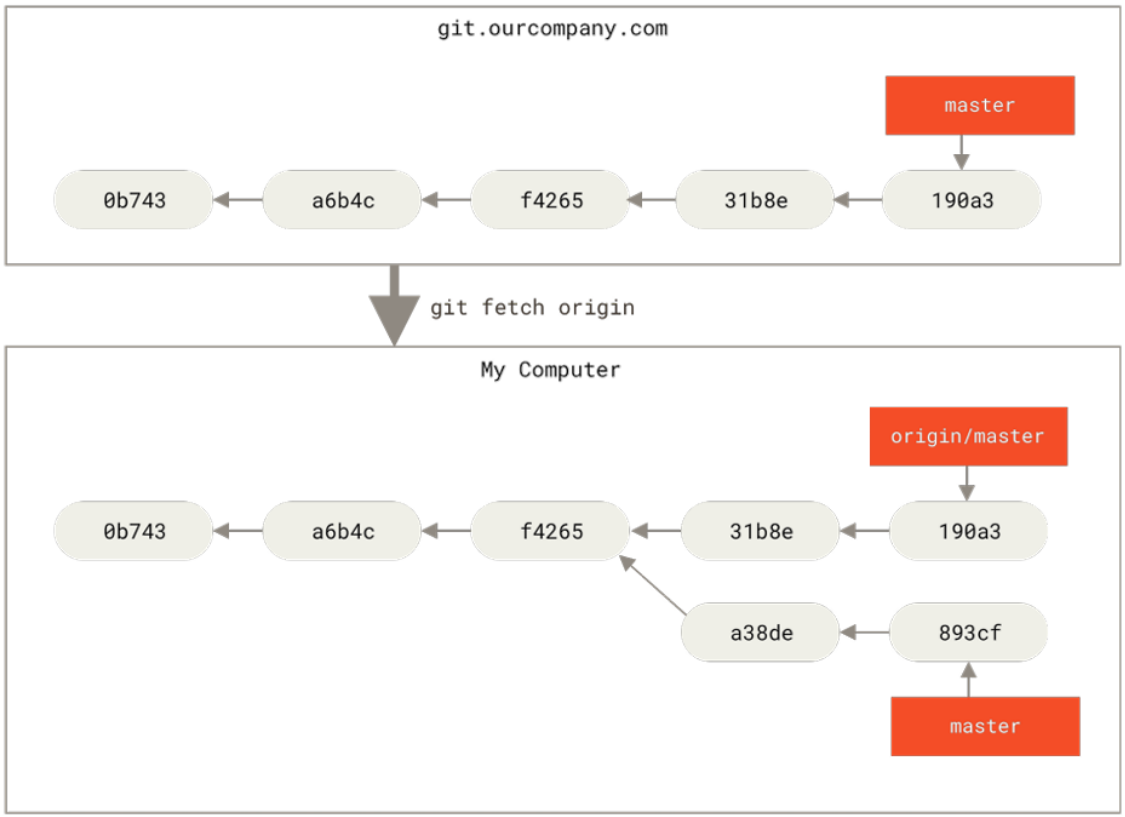
“origin” Is Not Special

- `origin` has no special meaning in Git, just like `master`
- It's simply the default name `git clone` assigns
- As histories diverge, `origin/master` won't move until you sync



Syncing with `git fetch`

- Command: `git fetch <remote>` (e.g., `git fetch origin`)
- Looks up the remote's URL
- Fetches data you don't yet have
- Moves your `origin/master` pointer to the new position



Introducing `git pull`

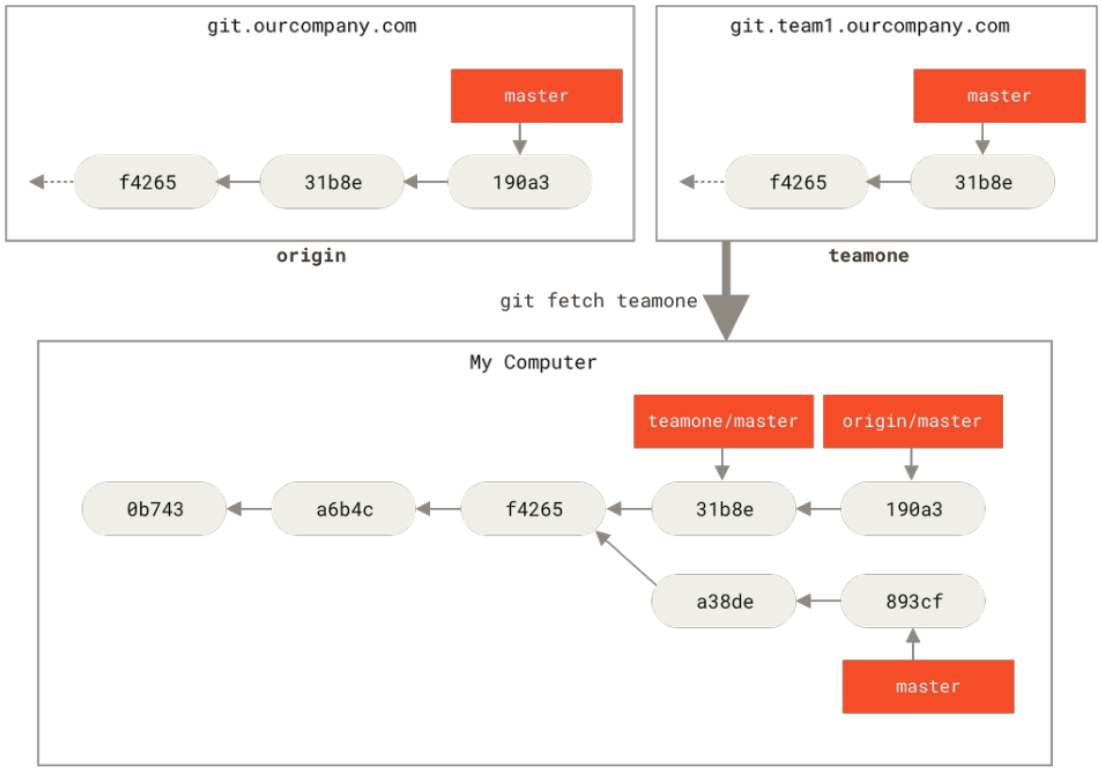
- If your branch tracks a remote branch, `git pull` can automate the workflow
- `git pull` = fetch + merge in a single command
- May be a more comfortable workflow for everyday use

`fetch` vs. `pull` at a Glance

- `git fetch`: downloads only — safe, no working directory changes
- `git pull`: downloads **and** merges (or rebases) automatically
- Use `fetch` first if you want to review changes before integrating
- Use `pull` for a fast, routine sync on a tracked branch

Multiple Remote Servers

- You can track more than one remote at a time
- Add a second remote: `git remote add teamone <url>`
- Fetch from it: `git fetch teamone`
- Creates a separate tracking branch: `teamone/master`



Pushing — Overview

- Local branches are **not** auto-synced to remotes
- You must explicitly push branches you want to share
- Enables private branches alongside shared topic branches

Pushing a Branch

- Command: `git push <remote> <branch>`
- Example: `git push origin serverfix`
- Meaning: "push my local serverfix to the remote's serverfix"

What Collaborators See

- After your push, collaborators run `git fetch origin`
- They get a new remote-tracking branch: `origin/serverfix`
- Fetching does **not** create an editable local copy automatically

Working from a Fetched Branch

- Merge it directly: `git merge origin/serverfix`
- Or create your own editable branch:
- `git checkout -b serverfix origin/serverfix`

Tracking Branches — Overview

- Checking out from a remote-tracking branch creates a "tracking branch"
- The branch it tracks is called the "upstream branch"
- On a tracking branch, `git pull` knows exactly what to fetch and merge
- Cloning auto-creates `master` tracking `origin/master`

Creating Tracking Branches

- Full form: `git checkout -b <branch> <remote>/<branch>`
- Shorthand: `git checkout --track origin/serverfix`
- Shortest form: `git checkout serverfix` (auto-detected if unambiguous)

Tracking Branch with a Different Local Name

- `git checkout -b sf origin/serverfix`
- Local branch `sf` now tracks `origin/serverfix`
- Pulling on `sf` will pull from `origin/serverfix`

Changing the Upstream Branch

- For an existing local branch: `git branch -u origin/serverfix`
- Also written as `--set-upstream-to`
- Resets which remote branch is tracked, at any time

Viewing Tracking Relationships

- Command: `git branch -vv`
- Shows each branch's upstream and ahead/behind counts
- Counts are only as fresh as your last fetch
- For live numbers: `git fetch --all; git branch -vv`

```
$ git branch -vv
  iss53     7e424c3 [origin/iss53: ahead 2] Add forgotten brackets
  master    1ae2a45 [origin/master] Deploy index fix
* serverfix f8674d9 [teamone/server-fix-good: ahead 3, behind 1] This should do it
  testing   5ea463a Try something new
```

Pulling

- `git fetch` downloads data but never touches your working directory
- `git pull` = `git fetch` + `git merge` in most cases
- On a tracking branch, `git pull` auto-detects server and branch

Deleting Remote Branches

- Use once a feature is merged and the branch is no longer needed
- Command: `git push origin --delete serverfix`
- Removes the pointer from the server (data often recoverable pre-GC)

Table of Contents

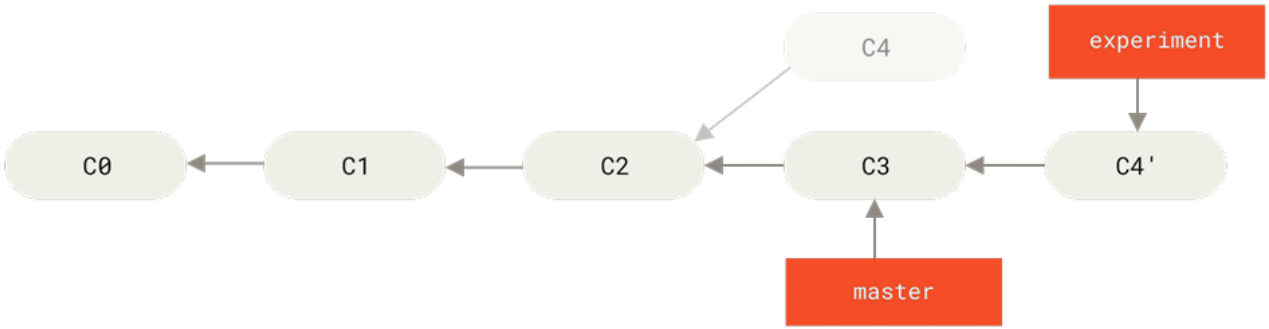
1. Branching
2. Branch Management
3. Remote Branches
4. Rebasing

The Basic Rebase — Concept

- Merge: three-way merge between two tips and their common ancestor
- Rebase: takes the patch from one branch and replays it on top of another
- Same end snapshot, different history shape

Performing a Basic Rebase

- `git checkout experiment`
- `git rebase master`
- Git rewinds, replays each commit's diff on top of `master`

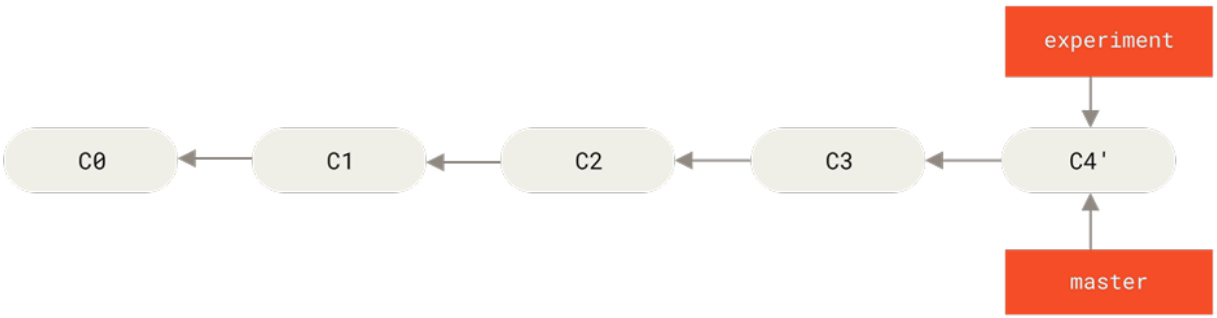


How Rebase Works Internally

1. Find the common ancestor of both branches
2. Compute the diff introduced by each commit on your branch
3. Save those diffs as temporary patches
4. Reset your branch to match the target branch
5. Reapply each saved patch in order

Fast-Forwarding After a Rebase

- `git checkout master`
- `git merge experiment`
- Since history is now linear, this is a simple fast-forward

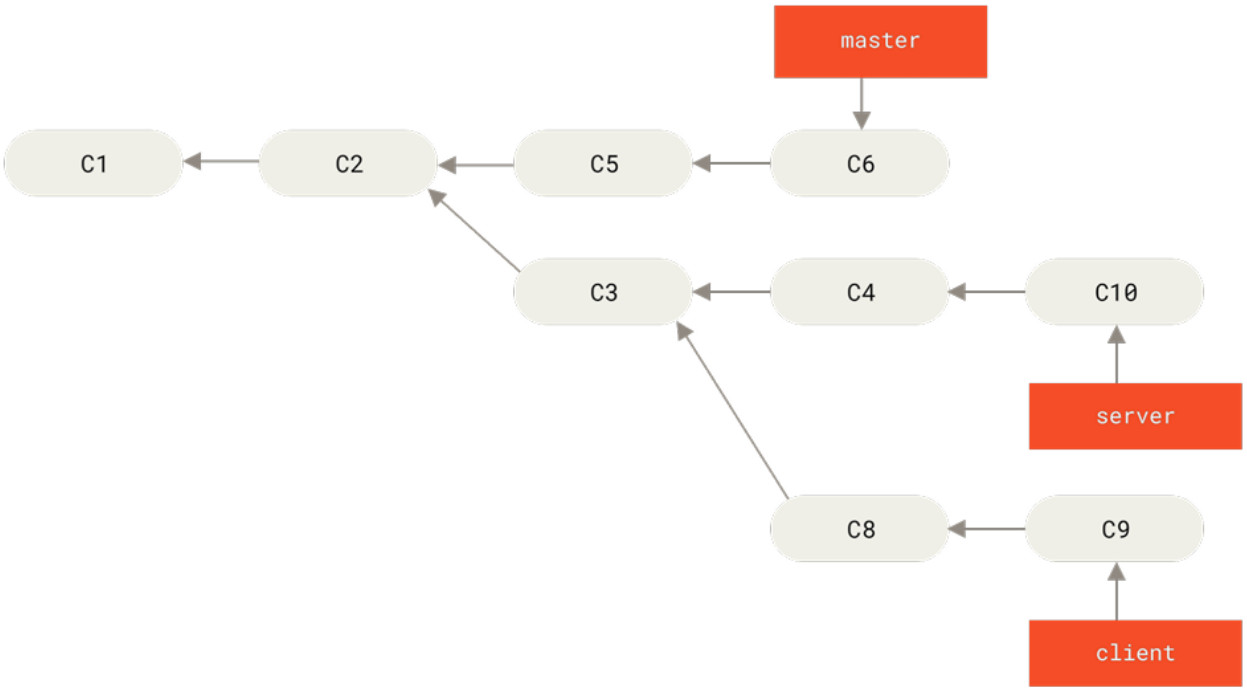


Rebase Result vs. Merge Result

- Final snapshot is identical either way
- Rebase produces a clean, linear-looking history
- Merge preserves the true parallel timeline
- Common use: rebase your work onto `origin/master` before submitting patches

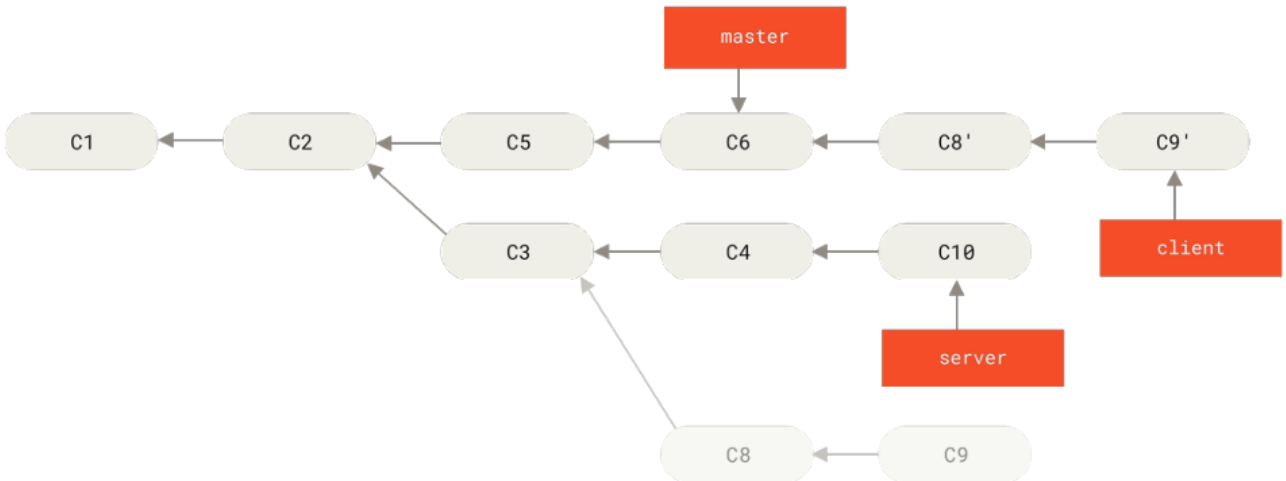
More Interesting Rebases: `--onto`

- Scenario: `client` branched off `server`, which branched off `master`
- Goal: merge only `client`'s changes, skip `server`'s for now
- Command: `git rebase --onto master server client`



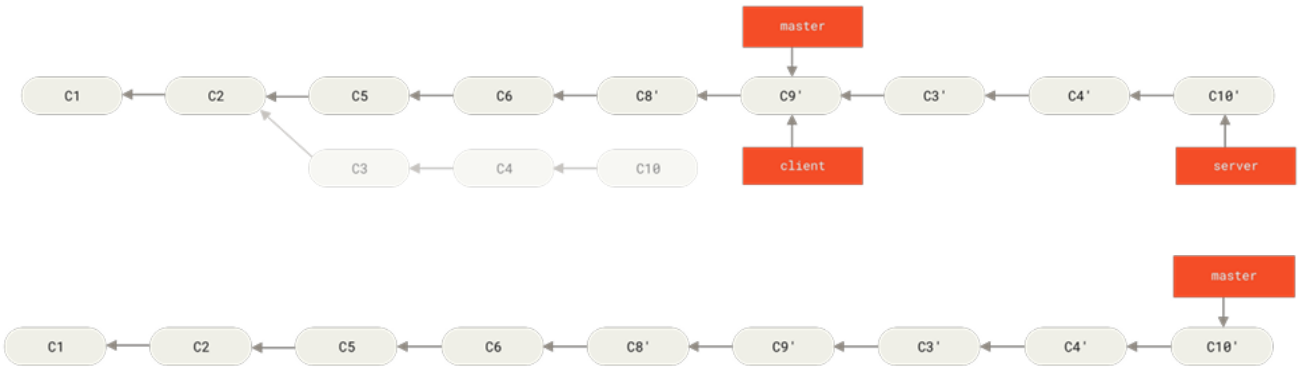
Understanding `--onto`

- Take the `client` branch
- Find patches since it diverged from `server`
- Replay those patches as if based directly on `master`
- Then fast-forward `master`: `git checkout master && git merge client`



Rebasing a Branch Without Checking It Out

- `git rebase <basebranch> <topicbranch>`
- Example: `git rebase master server`
- Checks out `server` for you and replays it onto `master`
- Then: `git checkout master && git merge server`



Cleaning Up After Integration

- Once `client` and `server` are fully integrated into `master`
- `git branch -d client`
- `git branch -d server`
- Result: a clean, linear final history

Choosing Between Rebase and Merge

- Neither approach is universally “correct”
- The right choice depends on your team and project
- Best-of-both-worlds compromise:
  - Rebase local, unpushed changes to clean up
  - Never rebase anything already pushed publicly